

Chapter 09

Text Splitters

Breaking large documents into small, meaningful chunks — the second core building block of any RAG pipeline, right after Document Loaders.

WHAT THIS CHAPTER COVERS

- ▶ What text splitting is and why LLM apps need it
- ▶ Length-Based Text Splitting (CharacterTextSplitter)
- ▶ Chunk size and chunk overlap
- ▶ Text Structure-Based Splitting (RecursiveCharacterTextSplitter)
- ▶ Document Structure-Based Splitting (code, Markdown)
- ▶ Semantic Meaning-Based Splitting (SemanticChunker)
- ▶ Connecting Document Loaders with Text Splitters
- ▶ Trade-offs between all four splitting strategies

Text Splitters

RECAP — CHAPTER 8

The previous chapter covered **Document Loaders** — the first RAG component, responsible for pulling raw content into LangChain from sources like PDFs, websites, and databases as standardized **Document** objects. This chapter picks up right where that left off: once a document is loaded, it usually needs to be broken into smaller pieces before it can be processed effectively.

1. Introduction to Text Splitting

Definition

Text Splitting is the process of breaking large pieces of text — articles, PDFs, HTML pages, books — into smaller, manageable pieces (chunks) that an LLM can handle effectively. The code/utility that performs this operation is called a *Text Splitter*.

Why It Exists

Working with very large blocks of text in a single LLM call is almost always a bad idea. If an entire multi-thousand-page PDF is fed to an LLM at once, the quality of the output degrades noticeably. It is a well-established practice in LLM application development to always break large text into smaller chunks before feeding it to a model — this consistently improves output quality.

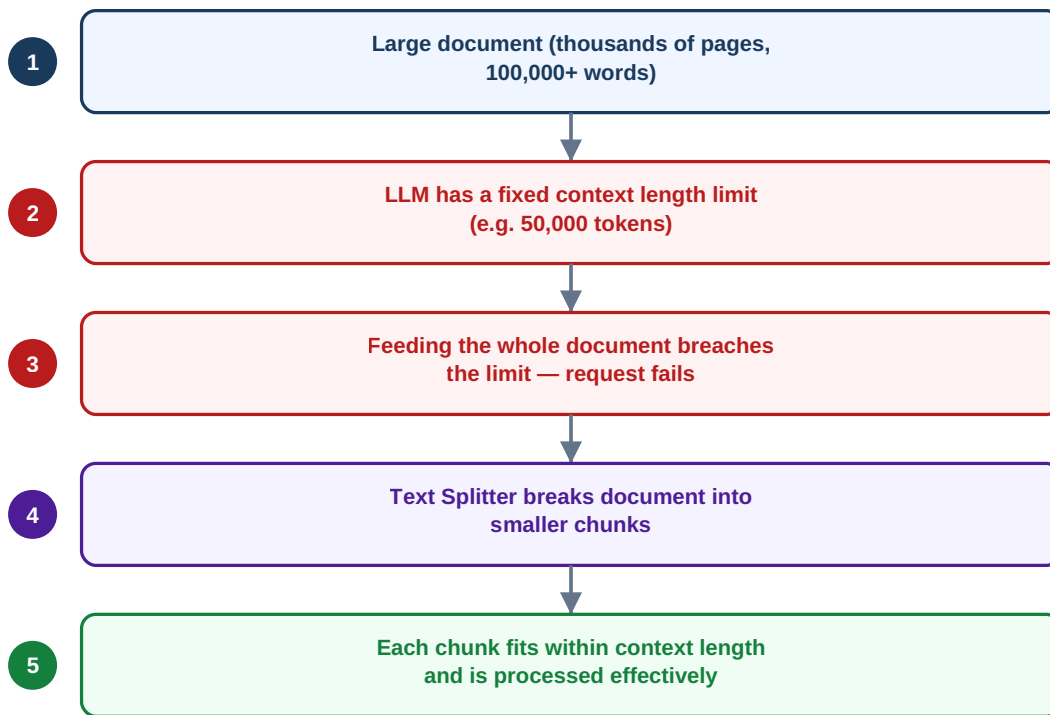


Figure 9.1 — The context-length problem that text splitting solves.

REASON 1: OVERCOMING THE MODEL'S CONTEXT LENGTH LIMIT

Every LLM has a fixed *context length* — the maximum amount of text (measured in tokens) it can accept as input in one go. For example, if a model's context length is 50,000 tokens and a source PDF contains over 100,000 words, that PDF simply cannot be sent to the model in a single request — the request would breach the context window. Text splitting solves this by dividing the document into chunks that individually fit within the limit.

REASON 2: BETTER RESULTS ON DOWNSTREAM TASKS

Many tasks built on top of LLMs — embeddings, semantic search, and summarization — empirically perform better on smaller chunks than on one giant block of text:

Downstream Task	Problem With a Single Large Block	Improvement From Chunking
Embeddings	An embedding model compresses the entire text into one vector; a large text covering many different topics cannot be captured well by a single vector — the semantic meaning gets diluted.	Each smaller chunk (e.g. one paragraph per IPL team) gets its own vector, which represents its specific meaning far more accurately.
Semantic Search	Searching against one enormous embedding gives low-precision, poor-quality matches.	Searching across many focused chunk embeddings returns a much more precise, relevant match for a user query.
Summarization	LLMs summarizing very large documents tend to drift off-topic or hallucinate details not present in the source.	Smaller, well-scoped chunks lead to noticeably more accurate and grounded summaries.

IMPORTANT

The embedding quality problem is intuitive: if a paragraph discusses CSK, then Mumbai Indians, then RCB, one single embedding vector cannot represent all three topics well. Splitting that paragraph into three separate chunks — one per team — and embedding each individually produces vectors that capture each team's semantic meaning far more precisely.

REASON 3: OPTIMIZING COMPUTATIONAL RESOURCES

Processing many small chunks is more memory-efficient than processing one large block of text, and it also allows chunks to be processed in parallel, further improving throughput.

THE FOUR TEXT SPLITTING STRATEGIES

This chapter covers four distinct text splitting techniques, in the order LangChain typically introduces them:

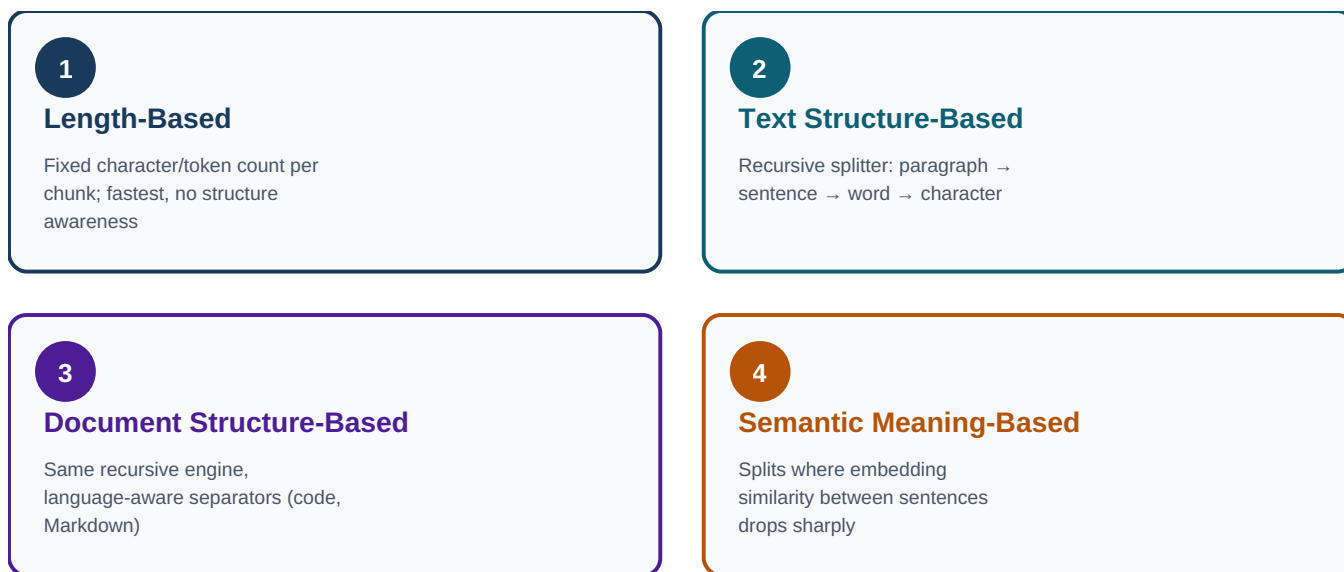


Figure 9.2 — The four text splitting strategies covered in this chapter.

🔄 Quick Revision — Introduction to Text Splitting

- Text splitting breaks large text into smaller, manageable chunks that an LLM can process effectively.
- Reason 1: LLMs have a fixed context length limit that large documents can easily exceed.
- Reason 2: Embeddings, semantic search, and summarization all produce better results on smaller chunks.
- Reason 3: Smaller chunks are more memory-efficient and can be processed in parallel.
- Four strategies exist: length-based, text structure-based, document structure-based, and semantic meaning-based.

2. Length-Based Text Splitting

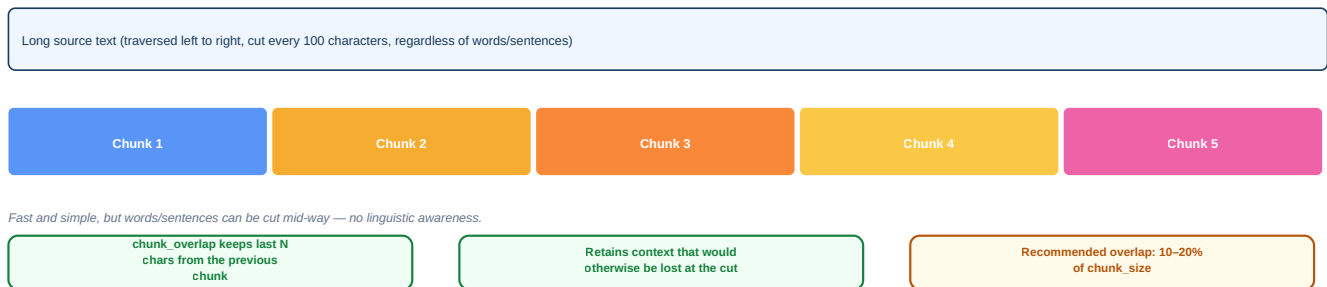
Definition

Length-Based Text Splitting is the simplest and fastest way to split text. A fixed chunk size is decided in advance (measured in characters or tokens), and the text is traversed from the start; as soon as the chunk size is reached, a cut is made and a new chunk begins.

How It Works

1. Decide a chunk size upfront (e.g. 100 characters).
2. Traverse the text from the beginning, counting characters (or tokens).
3. As soon as the count hits the chunk size, cut the text there to form a chunk.
4. Resume from that exact cut point and repeat until the entire text is consumed.

LENGTH-BASED SPLITTING (chunk_size = 100, chunk_overlap = 0)



Fast and simple, but words/sentences can be cut mid-way — no linguistic awareness.

Figure 9.3 — Fixed-size character chunking, with the role of chunk overlap.

COMMON MISTAKE

This method does not look at linguistic structure, grammar, or semantic meaning at all — it simply stops wherever the character count runs out. As a result, words, sentences, and paragraphs frequently get cut in the middle. This is fine for speed but can hurt semantic quality of downstream embeddings.

Example (Code)

```
from langchain_text_splitters import CharacterTextSplitter
from langchain_community.document_loaders import PyPDFLoader

# Load the source PDF (the interview-prep notes document used in this course)
loader = PyPDFLoader("Interview_notes.pdf")
docs = loader.lazy_load()

# chunk_size: max characters per chunk | chunk_overlap: shared chars between chunks
# separator = ' ' means: cut strictly by character count, ignore all text structure
splitter = CharacterTextSplitter(
    chunk_size = 300,
    chunk_overlap = 0,
    separator = ' '
)

# split_documents() works directly on loaded Document objects, not raw strings
result = splitter.split_documents(docs)
print(result)
```

This example connects the Document Loader from the previous chapter directly to a Text Splitter: `PyPDFLoader` loads `Interview_notes.pdf` into a list of per-page `Document` objects, and `split_documents()` then chunks every one of those `Document` objects into 300-character pieces with no overlap. Because `separator=' '`, the splitter cuts strictly on character count and pays no attention to word or sentence boundaries.

Chunk Overlap

Chunk overlap defines how many characters at the end of one chunk are repeated at the start of the next chunk. Instead of starting the next chunk exactly where the previous one ended, the splitter rewinds slightly and re-includes a bit of the previous chunk's content.

WHY IT EXISTS

Since length-based splitting can cut a sentence or thought abruptly in half, some context can be lost right at the boundary between two chunks. Chunk overlap preserves a small amount of shared context across the cut, so that meaning is not abruptly lost between one chunk and the next.

BEST PRACTICE

A commonly recommended overlap for RAG-based applications is **10–20% of the chunk size**. For example, with a chunk size of 100, an overlap of 10–20 characters is a reasonable starting point. Too little overlap risks losing context at the boundary; too much overlap increases the total number of chunks and therefore the computation required.

🔄 Quick Revision — Length-Based Text Splitting

- Length-based splitting divides text purely by a fixed character (or token) count.
- It is the fastest and simplest splitting method, implemented via `CharacterTextSplitter`.
- Its biggest drawback: no awareness of words, sentences, or grammar — text is cut wherever the count runs out.
- `chunk_overlap` repeats some characters from the end of one chunk at the start of the next to preserve context.
- A good default overlap is 10–20% of the `chunk_size`, balancing context retention against extra computation.
- `split_documents()` applies a splitter directly to `Document` objects returned by a loader like `PyPDFLoader`.

3. Text Structure-Based Text Splitting

Definition

Text Structure-Based Splitting takes advantage of the fact that text is inherently organized in a hierarchy: documents are made of paragraphs, paragraphs are made of sentences, sentences are made of words, and words are made of characters. The most widely used implementation of this idea in LangChain is the [*RecursiveCharacterTextSplitter*](#).

How It Works

The splitter is configured with an ordered list of separators, tried one after another:

RECURSIVE CHARACTER TEXT SPLITTER — SEPARATOR HIERARCHY

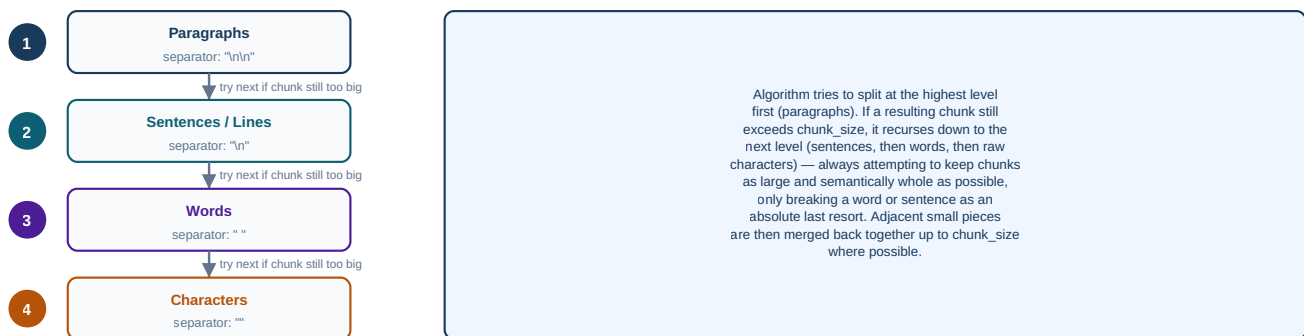


Figure 9.4 — Separator hierarchy used by *RecursiveCharacterTextSplitter*.

The algorithm always attempts the highest-priority separator first (paragraphs). If a resulting piece is still larger than the allowed `chunk_size`, it recurses down to the next separator level (sentences), then words, and finally raw characters as a last resort. After breaking things down, it also tries to **merge** adjacent small pieces back together, as long as the merged size stays within the allowed chunk size — this keeps chunks as close to the target size as possible without unnecessarily fragmenting text.

IMPORTANT

Unlike length-based splitting, this technique tries, at every level, to avoid cutting in the middle of a word or sentence for as long as possible. It only falls back to character-level splitting if nothing else produces a small-enough chunk.

Example (Code)

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

text = """Space exploration has led to incredible scientific discoveries. From landing on
the Moon to
exploring Mars, humanity continues to push the boundaries of what's possible beyond our
planet.

These missions have not only expanded our knowledge of the universe but have also
contributed to
advancements in technology here on Earth. Satellite communications, GPS, and even certain
medical
imaging techniques trace their roots back to innovations driven by space programs."""

# Default separators used internally: ["\n\n", "\n", " ", ""]
splitter = RecursiveCharacterTextSplitter(
    chunk_size = 100,
    chunk_overlap = 0
)

chunks = splitter.split_text(text)
print(len(chunks))
print(chunks)
```

With `chunk_size=100`, the splitter first tries to split on paragraph breaks (`\n\n`); since each paragraph here is still larger than 100 characters, it recurses down to sentence-level splitting. The behavior scales predictably with chunk size: a small chunk size (e.g. 10) forces splitting down to the word or character level, while a larger chunk size (e.g. 50 or above) lets whole paragraphs be kept together as single chunks whenever they fit.

INTERVIEW TIP

If asked to compare `CharacterTextSplitter` and `RecursiveCharacterTextSplitter`, lead with this: the former blindly counts characters and ignores structure entirely, while the latter recursively tries progressively finer separators (paragraph → sentence → word → character) so that splits happen at the most natural boundary possible for the given chunk size. This is why `RecursiveCharacterTextSplitter` is the most commonly used text splitter in practice.

🔄 Quick Revision — Text Structure-Based Text Splitting

- This technique exploits the natural hierarchy of text: paragraphs → sentences → words → characters.
- `RecursiveCharacterTextSplitter` tries separators in order: `\n\n`, then `\n`, then space, then empty string.
- It recurses to a finer separator only if the current chunk still exceeds `chunk_size`.
- After splitting, adjacent small pieces are merged back together up to the `chunk_size` limit where possible.
- It is the most widely used splitter in LangChain because it avoids mid-word/mid-sentence cuts wherever possible.

4. Document Structure-Based Text Splitting

Definition

Document Structure-Based Splitting is an extension of the same recursive splitting idea, applied to documents that are **not** plain natural-language text — such as source code or Markdown files. These documents are not organized into paragraphs and sentences; instead, they follow the syntax rules of their own format (classes, functions, loops for code; headings, lists for Markdown).

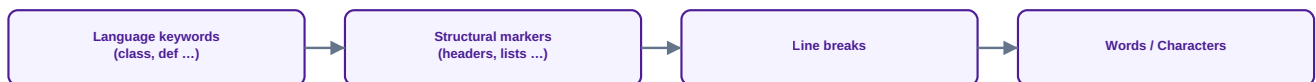
Why It Exists

Splitting a Python file or a Markdown document using plain-text separators (paragraph/sentence/word) makes no sense, since those documents are not structured that way. LangChain solves this by keeping the same `RecursiveCharacterTextSplitter` engine, but swapping in a language-aware set of separators via the `from_language()` constructor and a `Language` enum.

PLAIN TEXT



STRUCTURED DOCUMENTS (Python code, Markdown, etc.)



Same `RecursiveCharacterTextSplitter` engine — only the separator set changes, via `Language.PYTHON` / `Language.MARKDOWN` etc.

Figure 9.5 — Plain text vs. structured document separators.

Example — Splitting Python Code

```
from langchain_text_splitters import RecursiveCharacterTextSplitter, Language

# from_language() swaps in Python-specific separators: class, def, \n\n, \n, " ", ""
splitter = RecursiveCharacterTextSplitter.from_language(
    language = Language.PYTHON,
    chunk_size = 300,
    chunk_overlap = 0
)

code = """
from langchain_huggingface import ChatHuggingFace, HuggingFaceEndpoint
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import StrOutputParser
from dotenv import load_dotenv

load_dotenv()

llm = HuggingFaceEndpoint(
    repo_id = "Qwen/Qwen2.5-7B-Instruct",
    task = "text-generation"
)

model = ChatHuggingFace(llm = llm)

#1st Prompt - Detailed Report
template1 = PromptTemplate(
    template = "Write a detailed report on {topic}.",
    input_variables = ["topic"]
)

#2nd Prompt - Summary
template2 = PromptTemplate(
    template = "Write the summary on the following text /n {text} \n in 5 lines.",
    input_variables = ["text"]
)

parser = StrOutputParser()
chain = template1 | model | parser | template2 | model | parser

result = chain.invoke({"topic": "Black Holes"})
print(result)"""

chunks = splitter.split_text(code)
print(len(chunks))
print(chunks[1])
```

Because `Language.PYTHON` is used, the splitter's separator list is Python-aware: it prefers to break before top-level constructs (e.g. between import blocks, around function/class boundaries) rather than mid-statement. Note this example uses `HuggingFaceEndpoint` with `Qwen/Qwen2.5-7B-Instruct` wrapped in `ChatHuggingFace` as the underlying model.

Example — Splitting Markdown

```
from langchain_text_splitters import RecursiveCharacterTextSplitter, Language

# from_language() swaps in Markdown-specific separators: headings, lists, \n\n, \n, " ", ""
splitter = RecursiveCharacterTextSplitter.from_language(
    language = Language.MARKDOWN,
    chunk_size = 150,
    chunk_overlap = 0
)

code = """
# Smart Campus Food Ordering System

A multi-campus food ordering and outlet management platform designed to reduce wait
times, manage peak-hour load, and optimize campus food operations.

## Status
    Under active development (Phase 1: Core System)

## Vision
Single-campus deployment initially, architected for multi-campus scalability."""

chunks = splitter.split_text(code)
print(len(chunks))
print(chunks[1]) # e.g. the "## Status" section as its own chunk
```

With Markdown-aware separators, the splitter naturally prefers to break along heading boundaries (`#`, `##`) before falling back to blank lines or plain line breaks, so each resulting chunk tends to align with a logical section of the document rather than an arbitrary character count.

BEST PRACTICE

`from_language()` supports many languages beyond Python and Markdown — including JavaScript, Java, PHP, and HTML — via the same `Language` enum. The underlying algorithm never changes; only the separator set does.

Quick Revision — Document Structure-Based Text Splitting

- This method handles documents that are not plain text — e.g. source code, Markdown.
- It reuses `RecursiveCharacterTextSplitter`, but with language-aware separators via `from_language()`.
- `Language.PYTHON` splits around class/def boundaries before falling back to lines, words, characters.
- `Language.MARKDOWN` splits around heading and list boundaries before falling back to lines/words.
- The same approach extends to JavaScript, Java, PHP, HTML, and other supported languages.

5. Semantic Meaning-Based Text Splitting

Definition

Semantic Meaning-Based Splitting decides where to split not based on length or structure, but based on the actual **meaning** of the text. It aims to detect when the topic changes and place a chunk boundary exactly there.

Problem It Solves

Both length-based and text-structure-based splitting can fail on a specific kind of input: a single paragraph that actually contains two (or more) completely unrelated topics. For example, one paragraph might begin by discussing farmers and agriculture, then shift midway to talking about the IPL, followed by a second paragraph about terrorism. A structure-based splitter would only see two paragraphs and produce two chunks — but ideally this text should become **three** chunks, one per topic, since embedding a chunk that mixes agriculture and cricket produces a poor-quality vector for either topic.

How It Works

SEMANTIC MEANING-BASED SPLITTING — SLIDING WINDOW SIMILARITY

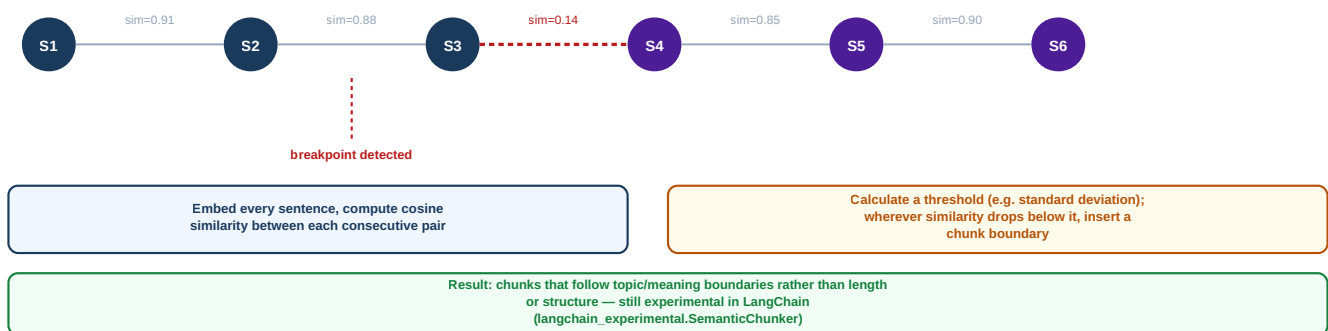


Figure 9.6 — Sliding-window sentence similarity used to detect topic breakpoints.

1. Split the text into individual sentences.
2. Generate an embedding vector for every sentence using an embedding model.
3. Compute the cosine similarity between each pair of consecutive sentence embeddings (sentence 1 vs. 2, sentence 2 vs. 3, and so on) using a sliding window.
4. Calculate a threshold from these similarity values (e.g. using standard deviation, percentile, interquartile range, or gradient-based methods).
5. Wherever the similarity between two consecutive sentences drops sharply below the threshold, mark that point as a topic change and insert a chunk boundary there.

Example (Code)

```
from langchain_experimental.text_splitter import SemanticChunker
from langchain_nvidia_ai_endpoints import NVIDIAEmbeddings
from dotenv import load_dotenv

load_dotenv()

# breakpoint_threshold_type: how the "sharp similarity drop" cutoff is calculated
# standard_deviation is one option among percentile / interquartile / gradient
splitter = SemanticChunker(
    NVIDIAEmbeddings(model="nvidia/nv-embedqa-e5-v5"),
    breakpoint_threshold_type = "standard_deviation",
    breakpoint_threshold_amount = 0.5
)

text = """
Farmers were working hard in the fields, preparing the soil and planting seeds for
the next season. The sun was bright, and the air smelled of earth and fresh grass.
The Indian Premier League (IPL) is the biggest cricket league in the world. People
all over the world watch the matches and cheer for their favourite teams.

Terrorism is a big danger to peace and safety. It causes harm to people and creates
fear in cities and villages. When such attacks happen, they leave behind pain and
sadness. To fight terrorism, we need strong laws, alert security forces, and support
from people who care about peace and safety."""

result = splitter.split_text(text)
print(result)
print(len(result))
```

Ideally, this text should produce three chunks — agriculture, IPL, and terrorism — since those are the three distinct topics present. In practice the boundaries are not always perfectly clean; a sentence about farmers can sometimes get pulled into the IPL chunk depending on the threshold chosen. Raising `breakpoint_threshold_amount` (e.g. to 3 standard deviations) increases tolerance for context change, which can cause the whole text to collapse into a single chunk if the threshold becomes too lenient.

Threshold Types

Threshold Type	What It Measures
Standard Deviation	Flags a breakpoint where the similarity drop exceeds N standard deviations from the mean similarity across the document.
Percentile	Flags a breakpoint at similarity drops below a given percentile of all observed similarity values.
Interquartile	Uses the interquartile range of similarity values to flag unusually large drops.
Gradient	Looks at the rate of change in similarity rather than the absolute drop, useful for gradual topic shifts.

COMMON MISTAKE

SemanticChunker is still an experimental component in LangChain (it lives in `langchain_experimental`, not the main library) and results are not always highly accurate in practice — sentences can end up grouped with the wrong topic. It should not yet be treated as a drop-in replacement for RecursiveCharacterTextSplitter in production systems.

REAL WORLD INSIGHT

As embedding models improve, semantic meaning-based splitting should become more reliable and more widely used. Among the four strategies in this chapter, **RecursiveCharacterTextSplitter remains the best all-round choice today** and is the one most commonly seen in real LangChain applications.

INTERVIEW TIP

If asked why SemanticChunker is not the default choice yet, point to two things: it lives in `langchain_experimental`, signalling it is not production-hardened, and every sentence requires its own embedding call, making it noticeably more expensive to run than the free, purely rule-based RecursiveCharacterTextSplitter.

Quick Revision — Semantic Meaning-Based Text Splitting

- Splits text based on meaning/topic rather than length or structural markers.
- Solves the case where a single paragraph mixes multiple unrelated topics.
- Works by embedding every sentence and computing cosine similarity between consecutive sentences.
- A sharp drop in similarity below a calculated threshold marks a topic-change breakpoint.
- Threshold types include standard deviation, percentile, interquartile range, and gradient.
- Implemented via the experimental SemanticChunker class; not yet as reliable or widely used as RecursiveCharacterTextSplitter.

K. Key Takeaways

- **Text Splitting** breaks large documents into smaller chunks so LLMs can process them effectively within their context length limits.
- **Length-Based Splitting** (CharacterTextSplitter) cuts text purely by character/token count — fast but structure-blind.
- **Chunk Overlap** repeats a small amount of text between consecutive chunks (typically 10–20% of chunk_size) to preserve context across cuts.
- **Text Structure-Based Splitting** (RecursiveCharacterTextSplitter) recursively tries paragraph → sentence → word → character separators to avoid unnatural mid-word cuts.
- **Document Structure-Based Splitting** reuses the same recursive engine with language-aware separators (via `from_language()`) for code, Markdown, and other structured formats.

- **Semantic Meaning-Based Splitting** (SemanticChunker) uses sentence-embedding similarity to detect topic changes, but remains experimental and less reliable today.
- Among all four strategies, **RecursiveCharacterTextSplitter** is the most widely used in production LangChain applications.

R. Revision Sheet

Text Splitting

Breaking large text into smaller chunks so LLMs and downstream tasks (embedding, search, summarization) work more effectively and accurately.

Length-Based Splitting

`CharacterTextSplitter` cuts strictly by `chunk_size`, ignoring linguistic structure entirely. Fastest but crudest method.

Chunk Overlap

Number of characters shared between consecutive chunks, used to retain context lost at a cut point. Recommended: 10–20% of `chunk_size`.

RecursiveCharacterTextSplitter

Tries separators in order (paragraph → sentence → word → character), recursing only when needed, then merges small pieces back up to `chunk_size`.

Document Structure-Based Splitting

Same recursive engine, language-aware separators via `from_language()` and the `Language` enum — supports Python, Markdown, JS, Java, PHP, HTML, and more.

Semantic Meaning-Based Splitting

`SemanticChunker` embeds sentences, computes consecutive cosine similarity, and splits where similarity drops sharply below a calculated threshold. Experimental.

Threshold Types (Semantic)

Standard deviation, percentile, interquartile range, and gradient — different ways of defining "how sharp a drop counts as a topic change."

I. Interview Questions

- Q1.** Why is text splitting necessary before feeding a document to an LLM, even if the model's context window is large enough to fit the whole document?
- Q2.** What are the main drawbacks of length-based (`CharacterTextSplitter`) text splitting?
- Q3.** Explain what `chunk_overlap` does and how you would decide a good overlap value for a given `chunk_size`.
- Q4.** Walk through how `RecursiveCharacterTextSplitter` decides where to split a piece of text.
- Q5.** Why can't you use the default paragraph/sentence/word separators to split source code or Markdown effectively?
- Q6.** How does document structure-based splitting differ internally from text structure-based splitting, given both use `RecursiveCharacterTextSplitter`?
- Q7.** Describe how semantic meaning-based splitting decides where to place a chunk boundary.
- Q8.** Give an example of a text input where length-based and text-structure-based splitting would both produce poor chunks, but semantic splitting would not.
- Q9.** What are breakpoint threshold types in `SemanticChunker`, and why might different threshold types produce different numbers of chunks?
- Q10.** Of the four splitting strategies covered, which is most commonly used in production LangChain applications today, and why?

F. Further Reading

- [LangChain documentation — Text Splitters overview](#) (`langchain_text_splitters`)
- [LangChain documentation — RecursiveCharacterTextSplitter and the `from\_language\(\)` API reference](#)
- [LangChain documentation — supported Language enum values](#) (Python, Markdown, JS, Java, HTML, and others)
- [LangChain Experimental documentation — SemanticChunker and breakpoint threshold types](#)
- [LangChain's interactive text-splitter visualization tool](#) (in the official docs) for hands-on experimentation with `chunk_size` and `chunk_overlap`